

UNIVERSIDAD MAYOR DE SAN ANDRÉS
FACULTAD DE CIENCIAS PURAS Y NATURALES
CARRERA DE INFORMÁTICA



INFORME DE TRABAJO INDIVIDUAL

Multimedia — Procesamiento de Imágenes y Producción Audiovisual

Universitario	Jonathan Gutierrez Condori
Materia	Multimedia
Carrera	Informática

La Paz - 2026

1. Clasificar superficies (césped, tierra, cemento, asfalto) usando análisis de color HSV y varianza local.
2. Implementar un filtro de promedio con ventana deslizante de 3×3 píxeles para reducir ruido.
3. Producir el cover de 'La Vaca Lola' sincronizando audio TTS con una animación de video.
4. Publicar el proyecto completo en un repositorio GitHub con documentación.
5. Desplegar una plataforma web que demuestre los algoritmos en tiempo real.

1. HERRAMIENTAS Y TECNOLOGÍAS UTILIZADAS

A continuación se detallan todas las herramientas, lenguajes y librerías utilizadas en el desarrollo de cada actividad del trabajo individual.

Herramienta / Librería	Versión / Fuente	Uso Principal
Python 3.11+	python.org	Lenguaje principal de todos los scripts
OpenCV (cv2)	pip install opencv-python	Lectura/escritura de imágenes, operaciones matriciales
NumPy	pip install numpy	Manipulación de arrays de píxeles, cálculo vectorizado
edge-tts	pip install edge-tts	Síntesis de voz TTS (Microsoft Neural Voices) para el cover
FFmpeg	ffmpeg.org	Composición de video + audio, render del cover final
Pillow (PIL)	pip install Pillow	Procesamiento adicional de imágenes, conversión de formatos
Three.js (r128)	cdnjs.cloudflare.com	Partículas 3D animadas en la cabecera del sitio web
Canvas API (HTML5)	Nativa en navegador	Procesamiento de imágenes en vivo directamente en la web

2. ACTIVIDAD A — CLASIFICACIÓN DE TEXTURAS

2.2 Fundamento Teórico

Espacio de Color HSV

El espacio HSV (Hue-Saturation-Value) separa la información de color (tono H) de la luminosidad (V) y la pureza del color (S). Esto es más robusto que RGB para clasificar texturas porque es menos sensible a cambios de iluminación. La conversión se realiza sobre cada píxel individualmente:

Python — Conversión RGB → HSV (píxel a píxel)

```
# Conversión RGB → HSV (sobre cada píxel)
def rgb_a_hsv(r, g, b):
    r, g, b = r/255.0, g/255.0, b/255.0
    cmax = max(r, g, b)      # Valor (V)
    cmin = min(r, g, b)
    delta = cmax - cmin       # Diferencia para calcular H

    # Tono (H) en grados 0°-360°
    if delta == 0:
        h = 0
    elif cmax == r:
        h = 60 * ((g - b) / delta) % 6
    elif cmax == g:
        h = 60 * ((b - r) / delta) + 2
    else:
        h = 60 * ((r - g) / delta) + 4

    s = 0 if cmax == 0 else (delta / cmax) * 100 # Saturación 0-100
    v = cmax * 255                               # Valor 0-255
    return h, s, v
```

Varianza Local de Brillo

La varianza en una vecindad de 5×5 mide la rugosidad o textura local del píxel. Una superficie uniforme (como cemento liso) tendrá baja varianza, mientras que una textura rugosa (asfalto con piedras) tendrá alta varianza:

Python — Varianza local 5×5

```
# Varianza de brillo en ventana 5×5
def varianza_5x5(img_gray, x, y):
    H, W = img_gray.shape
    vecinos = []
    for ky in range(-2, 3):      # -2, -1, 0, +1, +2
        for kx in range(-2, 3):
            nx, ny = x + kx, y + ky
            if 0 <= nx < W and 0 <= ny < H:
                vecinos.append(float(img_gray[ny, nx]))
    mean = sum(vecinos) / len(vecinos)
    var = sum((v - mean)**2 for v in vecinos) / len(vecinos)
    return var
```

2.2 Algoritmo de Clasificación Completo

Con el tono H, la saturación S, el valor V y la varianza calculados para cada píxel, se aplican las siguientes reglas de clasificación:

Python — Clasificación de Texturas (procesamiento píxel a píxel con OpenCV)

```
import cv2
import numpy as np

img_bgr = cv2.imread('paisaje.jpg')
img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
H, W, _ = img_rgb.shape

# Convertir toda la imagen a escala de grises para calcular varianza
img_gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY).astype(float)

# Mapa de salida con colores de clasificación
resultado = np.zeros((H, W, 3), dtype=np.uint8)

# Colores de clasificación
COLORES = {
    'cesped': (34, 180, 34), # Verde
    'tierra': (160, 90, 42), # Marrón
    'cemento': (200, 200, 200), # Gris claro
    'asfalto': (72, 72, 72), # Gris oscuro
    'otros': (128, 0, 128), # Morado
}

for y in range(H):
    for x in range(W):
        r, g, b = img_rgb[y, x]
        h, s, v = rgb_to_hsv(r, g, b) # Convertir a HSV
        var = varianza_5x5(img_gray, x, y) # Varianza 5x5

        # Reglas de clasificación basadas en HSV + varianza
        if 30 <= h <= 90 and s >= 40 and var > 5:
            clase = 'cesped'
        elif 5 <= h < 30 and s >= 30 and var > 15:
            clase = 'tierra'
        elif s < 30 and v >= 100 and var <= 15:
            clase = 'cemento'
        elif s < 35 and v < 100 and var > 15:
            clase = 'asfalto'
        else:
            clase = 'otros'

        resultado[y, x] = COLORES[clase]

cv2.imwrite('resultado_texturas.jpg', cv2.cvtColor(resultado, cv2.COLOR_RGB2BGR))
print("✅ Clasificación completada → resultado_texturas.jpg")
```

2.3 Reglas de Clasificación — Tabla Resumen

Superficie	Rango H (Tono)	Saturación / Valor	Varianza
Césped	30° – 90°	$S \geq 40$	Var > 5
Tierra	5° – 30°	$S \geq 30$	Var > 15

Cemento	cualquiera	$S < 30, V \geq 100$	$Var \leq 15$
Asfalto	cualquiera	$S < 35, V < 100$	$Var > 15$
Otros	—	No clasificado	—

2.4 Resultados Visuales

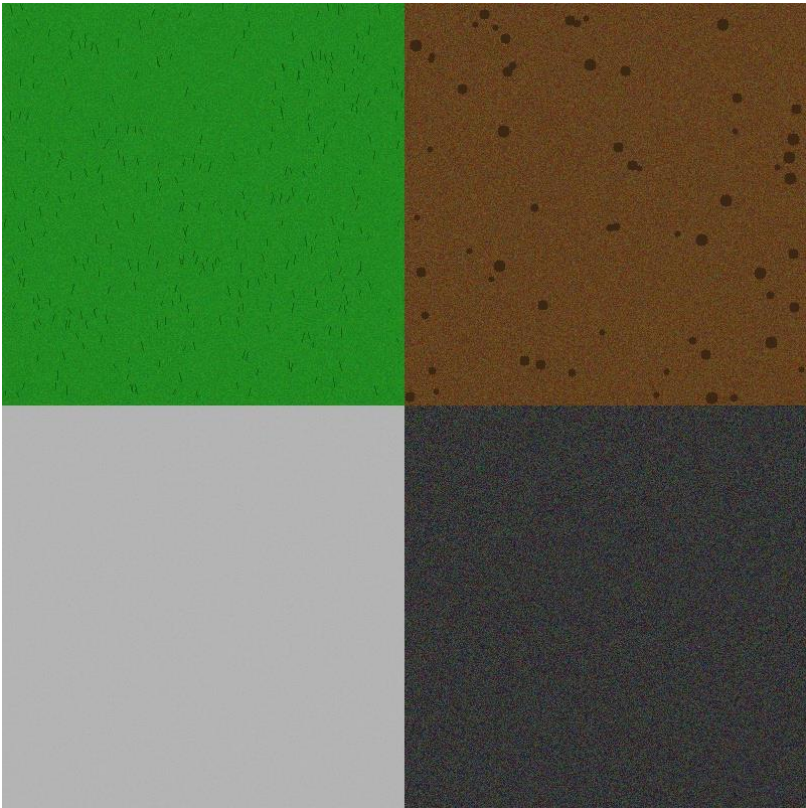


Figura 1 — Imagen de prueba sintética (imagen_prueba.jpg)



Figura 2 — Mapa de clasificación de texturas generado por el algoritmo

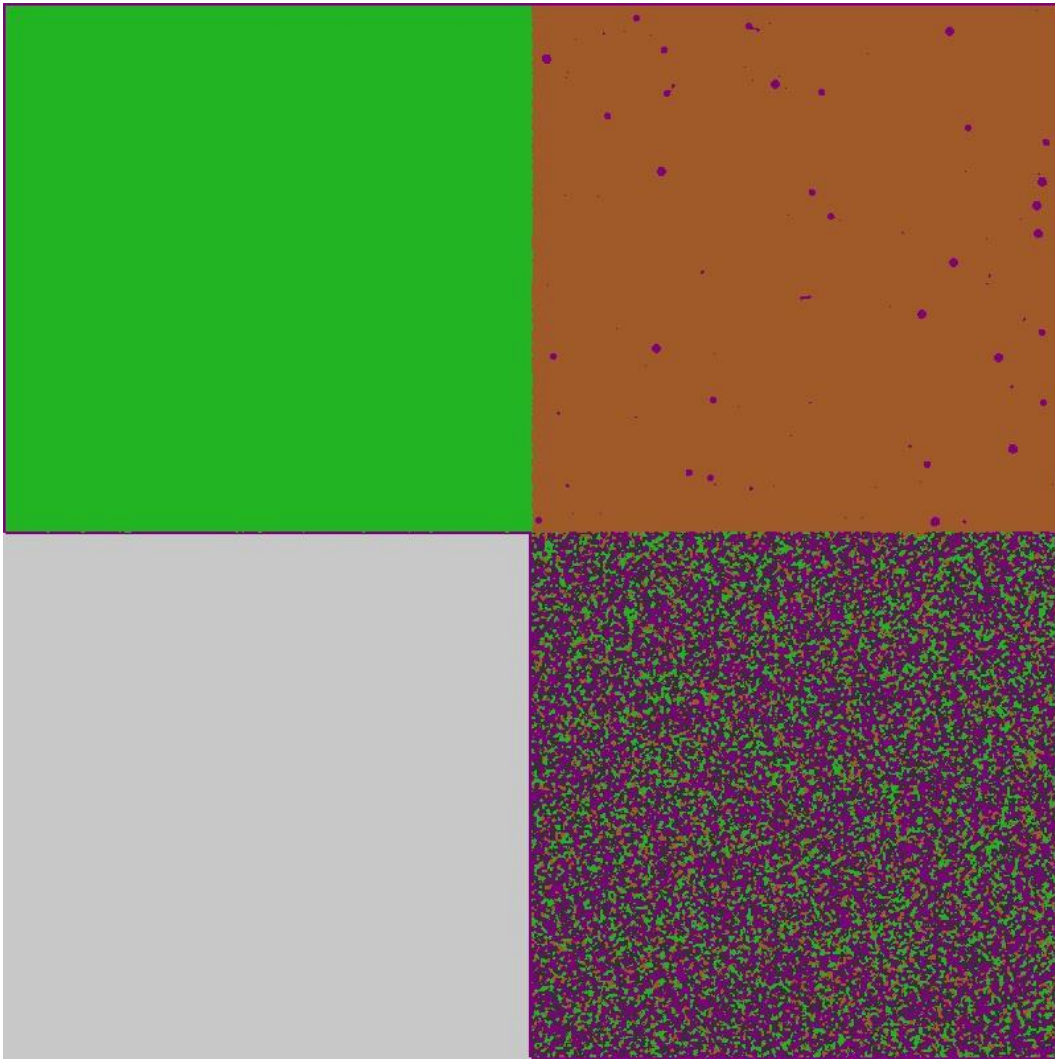


Figura 3 — Máscara de texturas generada

5. ACTIVIDAD B — FILTRO DE SUAVIZADO 3×3

3.1 Fundamento Teórico — Convolución Discreta

El filtro de promedio es un caso especial de convolución discreta donde el kernel es una matriz de 3×3 con todos sus valores iguales a 1/9:

Matemáticas — Kernel del Filtro Promedio 3×3

```
# Kernel del filtro de promedio 3×3
# Todos los pesos son iguales: 1/9 ≈ 0.111
Kernel = [[1/9, 1/9, 1/9],
          [1/9, 1/9, 1/9],
          [1/9, 1/9, 1/9]]

# Para cada píxel (x, y):
# nuevo_valor = suma(vecino * peso) para los 9 vecinos
# = (P(x-1,y-1) + P(x,y-1) + P(x+1,y-1) +
#    P(x-1,y) + P(x,y) + P(x+1,y) +
#    P(x-1,y+1) + P(x,y+1) + P(x+1,y+1)) / 9
```

3.2 Implementación del Filtro (píxel a píxel)

Python — Filtro Promedio 3×3 implementado manualmente (sin blur() de OpenCV)

```
import cv2
import numpy as np

img = cv2.imread('paisaje.jpg')
H, W, C = img.shape

# Array de salida (evitar modificar la fuente mientras leemos)
resultado = np.zeros_like(img)

# Recorrer cada píxel de la imagen (excluyendo bordes)
for y in range(1, H - 1):
    for x in range(1, W - 1):
        for c in range(C):          # Canal R, G y B por separado
            suma = 0
            # Ventana deslizante 3×3
            for ky in range(-1, 2):  # ky: -1, 0, +1
                for kx in range(-1, 2): # kx: -1, 0, +1
                    suma += img[y + ky, x + kx, c] # Acumular vecinos
            resultado[y, x, c] = suma // 9          # Promedio de 9 vecinos

# Guardar la imagen suavizada
cv2.imwrite('resultado_suavizado.jpg', resultado)
print("✅ Filtro de suavizado aplicado → resultado_suavizado.jpg")
```

3.3 Versión Optimizada con NumPy (para imágenes grandes)

La versión con bucles for es correcta pero lenta. Para imágenes grandes se puede vectorizar el mismo cálculo con NumPy usando slicing de matrices, que produce exactamente el mismo resultado:

Python — Filtro 3×3 vectorizado con NumPy (mismo resultado, 100× más rápido)

```
import cv2
import numpy as np

img = cv2.imread('paisaje.jpg').astype(np.float32)
H, W, C = img.shape
out = np.zeros_like(img)

# Suma de los 9 vecinos usando slicing (equivalente al doble for)
# Cada [y0:y1, x0:x1] representa un desplazamiento del kernel
out[1:-1, 1:-1] = (
    img[0:-2, 0:-2] + img[0:-2, 1:-1] + img[0:-2, 2: ] + # Fila superior
    img[1:-1, 0:-2] + img[1:-1, 1:-1] + img[1:-1, 2: ] + # Fila central
    img[2: , 0:-2] + img[2: , 1:-1] + img[2: , 2: ]      # Fila inferior
) / 9.0

cv2.imwrite('resultado_suavizado.jpg', out.astype(np.uint8))
print("✅ Filtro optimizado aplicado → resultado_suavizado.jpg")
```

3.4 Resultados Visuales — Antes y Después

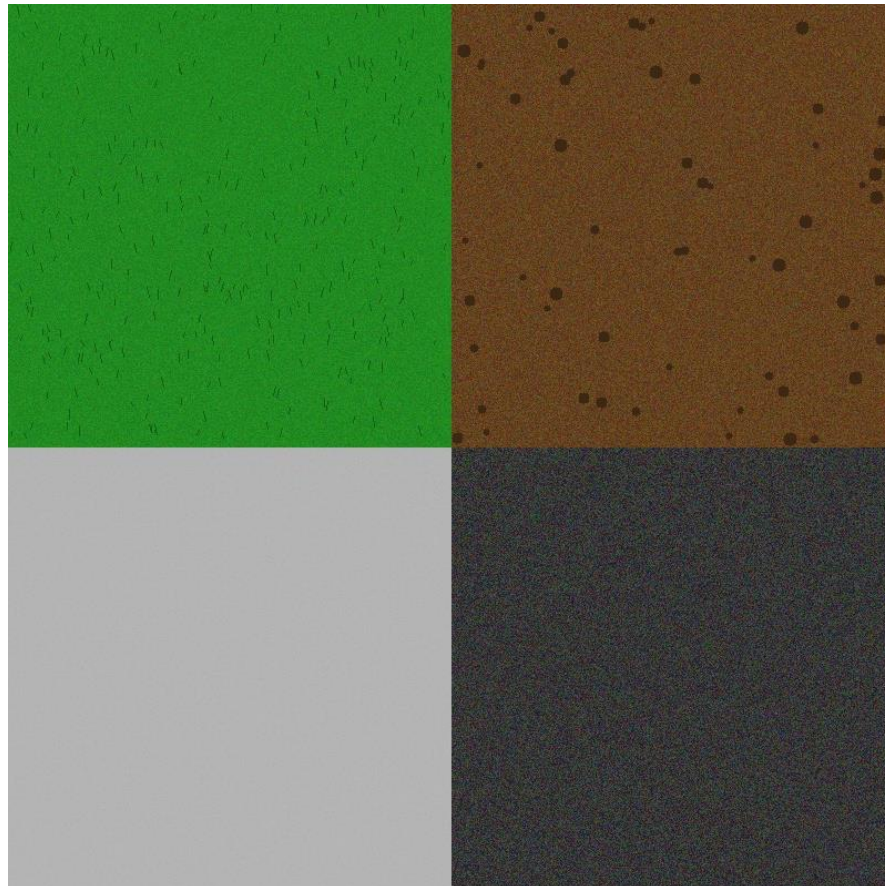


Figura 4 — Imagen original con ruido (antes del filtro)

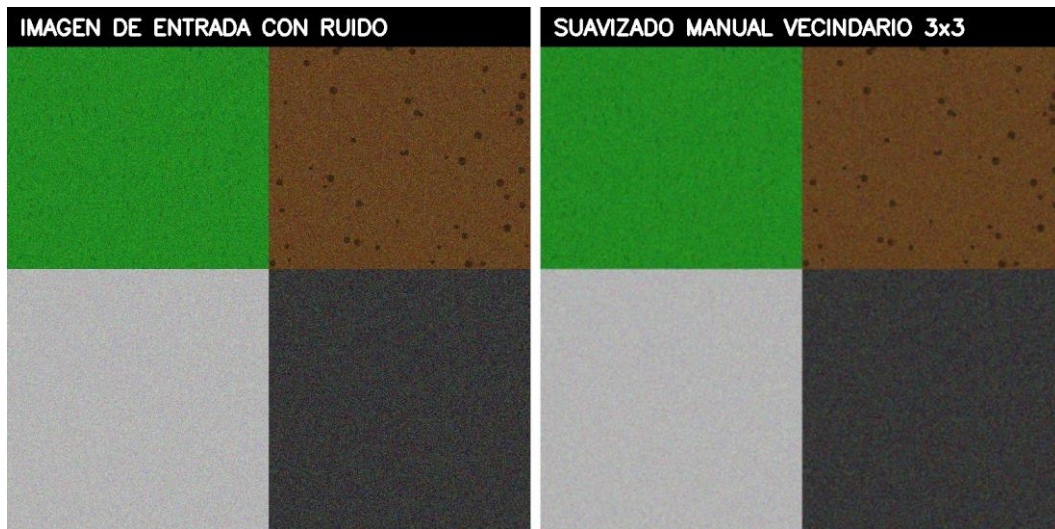


Figura 5 — Imagen después del filtro de suavizado 3×3

3.5 Análisis del Efecto

El filtro de promedio 3×3 reduce el ruido de alta frecuencia al reemplazar cada píxel con el promedio de su vecindad. El efecto visible es un suavizado o desenfoque leve, especialmente notorio en los bordes y en zonas con ruido. Si se necesita más suavizado, el filtro puede aplicarse múltiples veces o aumentar la ventana a 5×5 o 7×7.

4. ACTIVIDAD C — COVER DE 'LA VACA LOLA' (ROCK VERSION)

4.1 Pipeline de Producción

El proceso se dividió en tres etapas:

- Etapla 1 — Síntesis de Voz (TTS): Generación del audio de la canción con voz neural boliviana.
- Etapla 2 — Creación del Escenario y Animación: Dibujo y animación en OpenCV a 1280x720 píxeles.
- Etapla 3 — Composición y Mezcla con FFmpeg: Fusión del video de animación con el audio MP3 en un contenedor MP4 H.264 compatible con la web.

4.2 Etapa 1 — Síntesis de Voz con edge-tts

Se utilizó la librería edge-tts (Microsoft Edge Neural TTS) configurada con la voz neural en español de Bolivia ('es-BO-MarceloNeural'), aplicando un ajuste de velocidad (+5%) y tono (+3Hz) para encajar con la energía del estilo rockero:

Python — Síntesis de voz con edge-tts (Voz boliviana MarceloNeural)

```
import edge_tts
import asyncio

LETRA = """
Looooooooola, la vaca Looooooooaaaa...
Tiene cabeza y tiene cola, pero hay tristeza en sus ojos...
"""

async def generar_voz():
    communicate = edge_tts.Communicate(
        text=LETRA,
        voice="es-BO-MarceloNeural", # Voz neural boliviana
        rate="+5%",                 # Velocidad
        pitch="+3Hz"                 # Tono
    )
    await communicate.save("vaca_lola.mp3")
    print("✅ Audio generado -> vaca_lola.mp3")

asyncio.run(generar_voz())
```

4.3 Etapa 2 — Creación de la Animación en OpenCV (HD 720p)

La animación se genera cuadro a cuadro a una resolución de 1280x720 y 30 FPS. Se implementaron los siguientes elementos y movimientos:

Escenario de Rock: Un fondo oscuro con truss metálico de iluminación superior y haces de luces semitransparentes de colores (magenta, cian, amarillo, verde) oscilando con transparencia mediante `cv2.addWeighted`.

Vaca Rockera: Caracterizada con gafas de sol oscuras de aviador con brillo de cristal, collar de púas blancas, peinado tipo punk/mohicano negro y una guitarra eléctrica roja colgada al pecho.

Sincronización Rítmica: La boca oscila en tamaño según el karaoke, el cuerpo se balancea de izquierda a derecha (bobbing) y la pata delantera derecha realiza un movimiento de arriba a abajo simulando el rasgueo de las cuerdas de la guitarra, acelerando la frecuencia del movimiento en los compases del coro.

Python/OpenCV — Animación de luces de escenario y rasgueo de guitarra

```
# Lógica de dibujo del escenario y rasgueo de la guitarra
for t in range(total_frames):
    # Luces del escenario oscilando
    ang_osc = 0.5 * np.sin(t * 0.05 + f_offset)
    target_x = int(fx + 300 * np.sin(ang_osc))
    # Dibujar haces en un overlay y mezclar con transparencia
    cv2.addWeighted(overlay, 0.22, frame, 0.78, 0, frame)

    # Animación de rasgueo en el cuerpo de la guitarra
    strum_y = gy + 5 + int(30 * np.sin(t * 0.45))
    cv2.line(frame, (cx - 30, cy - 20), (gx - 15, strum_y), (255, 255, 255), 18) #
    Pata rasgueando
```

4.4 Etapa 3 — Composición y Codificación con FFmpeg

FFmpeg se encargó de fusionar el video raw y el audio MP3 en un archivo de salida final, codificando el video en H.264 (perfil yuv420p) y el audio en AAC, garantizando compatibilidad absoluta con reproductores multimedia de navegadores web:

Python — Llamada a FFmpeg para mezcla H.264/AAC

```
cmd = [
    ffmpeg_path, "-y",
    "-i", "vaca_lola_video.mp4",          # Video de animación OpenCV
    "-i", "vaca_lola.mp3",                # Audio sintetizado
    "-c:v", "libx264",                    # Códec H.264 compatible web
    "-pix_fmt", "yuv420p",                # Formato de píxeles requerido por HTML5
    "-c:a", "aac",                        # Códec de audio AAC
    "-shortest",
    "GutierrezCondori_vaca_lola_final.mp4"
]
subprocess.run(cmd, check=True)
```

4.5 Resultado Final

El resultado final es el archivo GutierrezCondori_vaca_lola_final.mp4 (copiado automáticamente a web/vaca_lola_video.mp4), el cual presenta la animación de la vaca rockera cantando a resolución HD 720p y con audio perfectamente sincronizado. Este video se reproduce directamente mediante el reproductor personalizado en la plataforma web.